



UNIVERSITE DE FIANARANTSOA

ECOLE NATIONALE D'INFORMATIQUE

MEMOIRE DE FIN D'ETUDES EN VUE D'OBTENTION DU DIPLOME DE
MASTER DE RECHERCHE

Mention : Intelligence Artificielle

Parcours : Gouvernance et Ingénierie des Données

Intitulée :

NATURAL LANGUAGE PROCESSING SUPPORT

Présenté par :

RAKOTOSON Hanitriniala Nicole

Encadreurs :

Dr DIMBIARISOA William Germain

Dr Stephane Ducasse

Année universitaire : 2024–2025

Remerciements

Je tiens à exprimer ma profonde gratitude à mes encadreurs pour leur accompagnement et leurs précieux conseils.

Table des figures

1.1	Algorithme de Viterbi	6
1.2	Structure de la RNN simple	13
1.3	La figure à gauche est une représentation compacte de la RNN simple et celle à droite est sa représentation déroulée dans le temps	14
1.1		

Table des matières

Remerciements	i
1 Traitement Automatique de Langage Naturel.	3
1.1 Importances de la NLP	3
1.1.1 Vie quotidienne.	3
1.1.2 Santé.	3
1.1.3 Entreprises.	4
1.1.4 Education.	4
1.2 Les algorithmes essentiels en TALN	4
1.2.1 Algorithmes de pretraitement.	4
1.2.2 Frequence des mots.	6
1.2.3 TF-IDF.	6
1.2.4 Skip-gram.	7
1.2.5 Régression logistique	8
1.2.6 Réseaux de neurones récurrents.	12
2 Byte Pair Encoding	18
2.1 Présentation.	18
2.1.1 Logique principale.	18
2.1.2 Illustration à travers un exemple.	19
2.1.3 Observation sur l'algorithme.	24
2.2 Implémentation.	25
2.2.1 Algorithme pseudo-code.	25
2.2.2 Implémentation en Pharo.	30
3 WordPiece	37
3.1 Présentation de WordPiece.	37
3.1.1 Comprendre WordPiece.	37
3.1.2 Logique de fonctionnement de WordPiece.	37
3.1.3 Illustration à travers un exemple.	38
3.2 Implémentation.	46
3.3 Présentation des algorithmes	46
3.4 Comparaison théorique	46

4	Implémentation	47
4.1	Implémentation de BPE	47
4.2	Implémentation de WordPiece	47
4.3	Implémentation de Unigram	47
5	Résultats et Analyse	48
5.1	Expérimentations	48
5.2	Comparaison des performances	48
6	Conclusion et Perspectives	49
A	Annexes	50

Introduction

Les machines ont toujours surpassé les humains dans les tâches de calcul numérique, mais elles ont longtemps été incapables d'égaliser nos capacités à comprendre ou à générer du texte. Aujourd'hui, grâce au TALN, les ordinateurs peuvent comprendre et générer du texte de manière automatisée.

Cependant les machines ne traitent pas directement le texte brut : celui-ci doit passer par plusieurs étapes de traitement avant d'être exploité par les modèles, notamment le prétraitement, l'extraction de caractéristiques, l'analyse et l'entraînement. La tokenisation, première étape du prétraitement, consiste à segmenter le texte en caractères, mots ou sous-mots. Bien que souvent négligée, elle influence directement la vitesse et la qualité de prédiction des modèles.

Parmi les méthodes les plus utilisées figurent BPE, WordPiece et Unigram. Ces algorithmes permettent de segmenter les mots en sous-unités, gérer les mots rares tout en conservant des séquences de longueur raisonnable et un vocabulaire de taille maîtrisée. Malgré leur disponibilité dans des bibliothèques comme Hugging Face Transformers et SentencePiece, ces algorithmes ne sont pas encore disponible dans Pharo.

Dans ce contexte, ce mémoire propose une implémentation naïve de ces trois algorithmes dans Pharo, suivie d'une optimisation et d'une étude comparative. L'objectif est de concevoir et d'implémenter les algorithmes BPE, WordPiece et Unigram, de réaliser un benchmarking de leurs performances en termes de vitesse et d'analyser la qualité des tokens générés. Cette démarche permet non seulement de combler un manque technique dans Pharo, mais aussi d'évaluer les différences entre ces méthodes.

Ce mémoire est organisé comme suit.

Le premier chapitre présente le contexte général du TALN et l'importance de la tokenisation.

Le deuxième chapitre introduit l'algorithme BPE ainsi que son implémentation naïve.

Le troisième chapitre présente l'algorithme WordPiece et son implémentation naïve.

Le quatrième chapitre est consacré à l'algorithme Unigram et à son implémentation naïve.

Le cinquième chapitre décrit l'optimisation de BPE, en présentant l'idée, la réalisation et les résultats du benchmarking.

Le sixième chapitre traite de l'optimisation de WordPiece, incluant la stratégie adoptée, l'implémentation et l'évaluation expérimentale.

Le septième chapitre présente l'optimisation de Unigram ainsi que son évaluation.

Enfin, le huitième chapitre propose une analyse comparative des tokens produits par les trois algorithmes sur les noms de méthodes en Pharo.

Chapitre 1

Traitement Automatique de Langage Naturel.

1.1 Importances de la NLP

Les données textuelles sont générées quotidiennement par de multiples sources, telles que les réseaux sociaux, les blogs, les forums, les articles en ligne, les communications professionnelles, les assistants virtuels, les oeuvres scientifiques et culturelles, ou encore les avis consommateurs. L'ensemble de ces générateurs produits un volume massif de textes difficile à exploiter manuellement, rendant le TALN indispensable pour transformer ces textes en informations utiles, exploitables et organisées.

1.1.1 Vie quotidienne.

Le NLP facilite l'interaction fluide entre l'homme et la machine. Grâce à lui, la barrière des langues peut être dépassée : les traducteurs automatiques comme Google Traduction permettent de comprendre et de communiquer dans différentes langues. Le NLP améliore également la recherche sur Internet, offrant résultats précis et adaptés. Les chatbots, applications du NLP, fournissent des réponses instantanées aux questions des utilisateurs à tout moment sans interruption. Enfin, le NLP permet la correction automatique de texte, le résumé de documents et la génération de contenus.

1.1.2 Santé.

Dans le secteur de la santé, le NLP analyse les symptômes des patients, leurs antécédants médicaux et les rapports de laboratoire afin d'aider au diagnostic. Il permet de lire automatiquement des milliers de dossiers médicaux, d'extraire des informations importantes et d'identifier des tendances pour assister les professionnels de santé. L'identification des effets indésirables aux médicaments à partir de données textuelles est une tâche complexe ; les modèles NLP peuvent scanner les dossiers des patients, les réseaux sociaux et la littérature médicale pour détecter d'éventuels effets secondaires ou interactions médicamenteuses.[1]

1.1.3 Entreprises.

De plus en plus d'entreprises utilisent le NLP pour automatiser certaines tâches, analyser les données et améliorer la gestion de relation client. Les systèmes NLP permettent de classifier automatiquement de grands volumes de documents. Ils sont également utilisés pour prédire les tendances boursières et les comportement des consommateurs en analysant les données issues des médias sociaux et de la presse, améliorant ainsi la satisfaction et la fidélisation des clients et orientant les stratégies marketing.

Le TALN enrichit l'informatique décisionnelle en exploitant les données non structurées, telles que les retours clients, les études de marché ou les réseaux sociaux. L'exploration de texte et les technologies du web sémantique sont utilisées pour améliorer l'analyse prédictive et la découverte de connaissances.[2]

1.1.4 Education.

Dans le domaine de l'éducation, le NLP personnalise les parcours d'apprentissage en analysant les réponses des étudiants, en identifiant leurs forces et leurs faiblesses, et en proposant des contenus adaptés à leurs besoins individuels. Il aide à améliorer la prononciation, la traduction et la compréhension des langues, et peut répondre aux questions des étudiants à tout moment via des chatbots. [3]

1.2 Les algorithmes essentiels en TALN

1.2.1 Algorithmes de pretraitement.

1.2.1.1 Tokenisation.

La tokenisation consiste à segmenter un texte brut en unités appelées tokens. Ces unités peuvent correspondre à des mots, des expressions, des phrases, des sous-mots ou même des caractères, selon la méthode de tokenisation adoptée. Il y a deux grandes classes d'algorithmes de tokenisation : Tokenisation descendante : cette approche repose sur la définition préalable de règles linguistiques ou de normes de segmentation. Le texte est ensuite découpé en appliquant ces règles, qui peuvent être basées sur des espaces, des signes de ponctuation.

exemple : la phrase : "Le chat dort." devient : ["Le", "chat", "dort", "."].

Tokenisation ascendante : cette approche s'appuie sur des méthodes statistiques appliquées aux séquences de caractères afin de construire un vocabulaire de sous-mots. Le texte est ensuite segmenté en fonction de ces unités. Les algorithmes tels que Byte Pair Encoding ou Word-Piece ou Unigram illustrent ce type d'approche. Nous allons détailler ces algorithmes dans les chapitres suivantes.

exemple : la phrase : "Le chat dort." devient : ["Le", "ch", "at", "dor", "t", "."]

1.2.1.2 Hidden Markov Model pour Part-Of-Speech Tagging.

Le HMM est un modele probabiliste de sequence. Etant donnee une sequence d'unites(mots, lettres, morphemes, etc), il calcule la probabilites de toutes les sequences possibles d'etiquettes et choisit la meilleure sequence d'etiquettes.

Le HMM est definit par cinq composants :

- Les etats Q : un ensemble d'etats $q_1, \dots, q_n$. Chaque etat correspond a une etiquette grammaticale.
- la matrice A contient les probabilites de passer d'un etat a un autre. Chaque valeur a_{ij} indique la probabilite de passer de l'etat i a l'etat j . La somme des probabilites de transition depuis un etat donne est egale à 1.
- Les observations O : la séquence de mots ou unités observables dans le texte. Chaque observation provient d'un vocabulaire donné.
- Les probabilites d'emissions B : elles indiquent la probabilité qu'un état génère une observation particulière.
- La distribution initiale π : elle donne la probabilité que la chaîne de Markov commence dans chaque état. Certains états peuvent avoir une probabilité initiale nulle s'ils ne peuvent pas commencer une séquence. La somme de toutes les probabilités initiales est égale à 1.

Pour pouvoir obtenir, la sequence d'etiquette correspondante a une sequence de mots nous allons proceder comme suit :

- Calculer les composantes A et B de HMM en comptant sur un corpus annote :

La matrice A contient les probabilites de transition entre etiquette $P(t_i|t_{i-1})$ qui representent la probabilite qu'une etiquette apparaisse etant donne l'etiquette precedente.

$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$ ou $C(t_{i-1}, t_i)$ est le nombre de fois ou l'etiquette t_{i-1} est suivi de t_i dans le corpus et $C(t_{i-1})$ est le nombre de fois ou l'etiquette t_{i-1} apparait.

B contient les probabilites d'emissions $P(w_i|t_i)$ representant la probabilite, etant donne une etiquette, qu'elle soit associee a un mot donne. $P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$ ou $C(t_i, w_i)$ est le nombre de fois ou le mot w_i apparait avec l'etiquette t_i dans le corpus annote et $C(t_i)$ est le nombre de fois ou l'etiquette t_i apparait.

- Apres avoir calculer A et B , nous allons maintenant determiner la sequence la plus probable en utilisant l'algorithme de Viterbi illustre ci-dessous.

```

function VITERBI(observations of len  $T$ , state-graph of len  $N$ ) returns best-path, path-prob

create a path probability matrix viterbi[ $N, T$ ]
create a backpointer matrix backpointer[ $N, T$ ]
for each state  $s$  from 1 to  $N$  do ; initialization step
    viterbi[ $s, 1$ ]  $\leftarrow \pi_s * b_s(o_1)$ 
    backpointer[ $s, 1$ ]  $\leftarrow 0$ 
for each time step  $t$  from 2 to  $T$  do ; recursion step
    for each state  $s$  from 1 to  $N$  do
        viterbi[ $s, t$ ]  $\leftarrow \max_{s'=1}^N$  viterbi[ $s', t-1$ ] *  $a_{s',s}$  *  $b_s(o_t)$ 
        backpointer[ $s, t$ ]  $\leftarrow \operatorname{argmax}_{s'=1}^N$  viterbi[ $s', t-1$ ] *  $a_{s',s}$  *  $b_s(o_t)$ 

    bestpathprob  $\leftarrow \max_{s=1}^N$  viterbi[ $s, T$ ] ; termination step
    bestpathpointer  $\leftarrow \operatorname{argmax}_{s=1}^N$  viterbi[ $s, T$ ] ; termination step
    bestpath  $\leftarrow$  the path starting at state bestpathpointer, that follows backpointer[] to states back in time
return bestpath, bestpathprob

```

Figure 17.10 Viterbi algorithm for finding the optimal sequence of tags. Given an observation sequence and an HMM $\lambda = (A, B)$, the algorithm returns the state path through the HMM that assigns maximum likelihood to the observation sequence.

FIGURE 1.1 – Algorithme de Viterbi

1.2.2 Frequence des mots.

Le modele Frequence des mots represente chaque document d'un corpus sous la forme d'un vecteur de dimension egale a la taille du vocabulaire¹. Chaque dimension du vecteur correspond a un mot du vocabulaire, et la valeur associee indique le nombre d'occurences de ce mot dans le document. Il ignore l'ordre des mots.

Supposons que notre corpus est compose des deux phrases suivantes :

P_1 = "Le chat mange le poisson." et P_2 = "Le poisson nage dans l'eau."

Dans ce cas, notre vocabulaire V est donne par :

{Le , chat, mange, le, poisson, nage, dans, l', eau }.

Afin de representer ces deux phrases sous forme vectorielle, on fixe un ordonne les mots du vocabulaire comme suit :

(Le, chat, mange, le, poisson, nage, dans, l', eau).

Ainsi, le vecteur correspondant a P_1 (respectivement P_2) est (1, 1, 1, 1, 1, 0, 0, 0, 0) (respectivement (1, 0, 0, 0, 1, 1, 1, 1, 1)).

1.2.3 TF-IDF.

Le TF-IDF est une methode d'extraction de caracteristiques de texte qui evalue l'importance d'un mot dans un document specifique par rapport a son occurence dans l'ensemble du corpus.

$$TF-IDF = TF(t, d) * IDF(t)$$

1. Le vocabulaire est l'ensemble des mots uniques du corpus

ou :

- $TF(t, d)$ est la fréquence d'apparition d'un mot t dans un document d spécifique.

$$TF(t, d) = \frac{\text{nombre total de mots dans le document } d}{\text{nombre d'occurrences du terme } t \text{ dans le document } d}$$

- $IDF(t, d)$ reflète l'importance de chaque terme dans un corpus de documents.

$$IDF(t) = \log\left(\frac{\text{nombre de documents contenant le terme } t \text{ dans le corpus}}{\text{nombre total de documents dans le corpus} + 1}\right)$$

Un TF-IDF élevé indique que le terme est fréquent dans le document mais rare dans l'ensemble du corpus.

1.2.4 Skip-gram.

Skip-gram est une méthode permettant de calculer des embeddings de mots, c'est-à-dire des représentations vectorielles denses et continues des mots.

À partir d'un corpus de documents, le modèle apprend un vecteur pour chaque mot du vocabulaire en entraînant un classificateur binaire. Étant donné une paire (w, c) , composée d'un mot cible w et d'un mot candidat c , ce classificateur estime la probabilité que c apparaisse dans le contexte de w . Les poids appris servent ensuite d'embeddings.

L'algorithme parcourt le corpus et considère chaque mot comme un mot cible. Pour chaque mot cible, il prend en compte ses voisins dans une fenêtre de contexte de taille L , qui détermine le nombre de mots à gauche et à droite à considérer.

Par exemple, dans la phrase : « ... lemon, a [tablespoon of apricot jam, a] pinch ... », si le mot cible est apricot et que $L = 2$, les mots de contexte sont tablespoon, of, jam et a.

Skip-gram apprend deux matrices :

- W : embedding des mots en tant que cibles.
- C : embedding des mots en tant que contextes.

Ces matrices sont initialisées aléatoirement puis ajustées au cours de l'entraînement.

Le modèle skip-gram apprend ainsi deux embeddings distincts pour chaque mot i : un embedding cible w_i et un embedding de contexte c_i , stockés respectivement dans les matrices W et C . En pratique, il est courant de combiner ces deux représentations en utilisant la somme $w_i + c_i$ comme embedding final du mot. Alternativement, on peut ne conserver que l'embedding cible w_i et ignorer la matrice C . Pour chaque paire positive² (w, c_{pos}) , on génère k paires négatives³ (w, c_{neg}) , où c_{neg} est un mot tiré aléatoirement selon une distribution unigramme pondérée :

$$P_a(w) = \frac{\text{count}(w)^a}{\sum_{w'} \text{count}(w')^a}. \text{ En pratique, on prend souvent } a = 0.75.$$

L'objectif est de rapprocher les embeddings des paires positives et d'éloigner ceux des paires négatives.

Pour atteindre cet objectif, les embeddings sont ajustés de manière itérative au cours de l'entraînement. Plus précisément, à chaque étape, les vecteurs sont mis à jour en fonction des paires positives et négatives.

2. Une paire positive est un couple composé de w , un mot cible c'est-à-dire le mot que l'on considère, et c_{pos} , un vrai mot qui apparaît dans la fenêtre de contexte autour de w .

3. Une paire négative est un couple composé de w , un mot cible c'est-à-dire le mot que l'on considère, et c_{neg} , un mot tiré aléatoirement dans le vocabulaire (différent de w).

Les équations de mise à jour, de l'instant t à $t + 1$, sont données par :

$$c_{pos}^{t+1} = c_{pos}^t - [s(c_{pos}^t \cdot w^t) - 1]w^t.$$

$$c_{neg}^{t+1} = c_{neg}^t - [s(c_{neg}^t \cdot w^t)]w^t.$$

$$w^{t+1} = w^t - ([s(c_{pos}^t \cdot w^t) - 1]c_{pos}^t + \sum_{i=1}^k [s(c_{neg}^t \cdot w^t)]c_{neg}^t).$$

$s(x)$ est la fonction sigmoïde :

$$s(x) = \frac{1}{1 + \exp(-x)}.$$

1.2.5 Régression logistique

La régression logistique est un classificateur discriminatif⁴, probabiliste et supervisé. Elle est utilisée pour classer des données en deux ou plusieurs catégories (par exemple, positif ou négatif). Dans ce cadre, le modèle est entraîné à partir d'un ensemble d'exemples étiquetés X , c'est-à-dire que chaque observation x est associée à une étiquette y .

Chaque observation x est décrite par un vecteur de caractéristiques $x = [x_1, \dots, x_f]$ où f est le nombre de caractéristiques. Celles-ci peuvent être conçues manuellement ou apprises automatiquement à partir des données, notamment via des méthodes d'apprentissage de représentations ou des gabarits de caractéristiques.

L'objectif de la régression logistique est de prédire la classe $\hat{y}$ d'une nouvelle observation x à partir de ces caractéristiques. Elle résout cette tâche en apprenant, à partir d'un ensemble d'entraînement, un vecteur de poids $w = [w_1, \dots, w_f]$ et un terme de biais b . Chaque poids est un nombre réel et est associé à l'une des caractéristiques. Il représente l'importance de cette caractéristique dans la décision de classification.

Comment un classificateur prédit-il la sortie $\hat{y}$ pour une observation x donnée, et comment apprend-il ses paramètres w et b (poids et biais) à partir d'un jeu d'entraînement X ? Supposons que $X \in \mathbb{R}^{m \times f}$ (une matrice de taille $m \times f$), où m est le nombre d'observations et f le nombre de caractéristiques. On peut intégrer le biais b dans le vecteur de poids w afin de simplifier les calculs lors de la prédiction et de l'entraînement du modèle. Cela conduit à définir un nouveau vecteur de poids $\tilde{w} \in \mathbb{R}^{f+1}$. Pour ce faire, on ajoute une colonne de 1 à la matrice X , ainsi qu'une composante égale à 1 à chaque vecteur de caractéristiques x . On obtient alors une nouvelle matrice $\tilde{X} \in \mathbb{R}^{m \times (f+1)}$ et un vecteur $\tilde{x} \in \mathbb{R}^{f+1}$.

1.2.5.1 Cas de la régression logistique binaire

Le but de la régression logistique binaire est d'entraîner un classificateur capable de prendre une décision binaire à partir d'une observation. Pour une observation donnée, le modèle produit une sortie $\hat{y} \in [0, 1]$, qui représente la probabilité que l'observation appartienne à une classe

4. Un classificateur discriminatif cherche à apprendre à distinguer les classes sans forcément modéliser leur génération.

donnée (classe 0). Une fois cette probabilité calculée, une décision binaire est obtenue en appliquant un seuil (par exemple 0,5) : si $\hat{y}$ est supérieure à ce seuil, l'observation est classée dans la classe 1, sinon elle est classée dans la classe 0.

Pour une observation x , le modèle calcule un score :

$$z = \tilde{w} \cdot \tilde{x}$$

Ce score est ensuite transformé en probabilité à l'aide de la fonction sigmoïde :

$$\hat{y} = s(z) = \frac{1}{1 + \exp(-z)}$$

La prédiction de la classe est obtenue en appliquant un seuil à $\hat{y}$.

Pour un jeu de données X , le modèle calcule un vecteur de scores linéaires :

$$z = \tilde{X} \cdot \tilde{w}^T$$

où $\tilde{w}^T$ est la tranposée de w et z est un vecteur de taille m .

Ces scores sont ensuite transformés en probabilités à l'aide de la fonction sigmoïde appliquée élément par élément :

$$\hat{y} = \sigma(z) = \left[\frac{1}{1 + e^{-z_1}}, \dots, \frac{1}{1 + e^{-z_m}} \right]$$

où y est un vecteur de taille m représentant la probabilité d'appartenance de chaque observation à une classe.

La prédiction de la classe est obtenue en appliquant un seuil (par exemple 0,5) à chaque composante de $\hat{y}$.

Pour que le modèle réalise de bonnes prédictions, il est nécessaire d'apprendre des paramètres adaptés (poids et biais). Pour cela, ces paramètres sont initialisés de manière aléatoire. Ensuite, on mesure l'erreur entre les prédictions $\hat{y}$ et les vraies valeurs y à l'aide d'une fonction de perte. La fonction de perte utilisée pour la régression logistique binaire est la fonction de perte d'entropie croisée :

$$L_{CE}(\hat{y}, y) = -[y \log(s(\tilde{w} \cdot \tilde{x})) + (1 - y) \log(1 - s(\tilde{w} \cdot \tilde{x}))]$$

Cette fonction de perte quantifie l'écart entre les prédictions du modèle et les valeurs réelles. Un algorithme d'optimisation, généralement la descente de gradient, est ensuite utilisé pour minimiser cette perte. Les paramètres sont alors ajustés progressivement de sorte que les prédictions du modèle se rapprochent de plus en plus des valeurs réelles :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta \nabla L(s(\tilde{w} \cdot \tilde{x}), y)$$

où :

— $L(\hat{y}, y)$ est la fonction de perte qui mesure la proximité entre la prédiction $\hat{y}$ et la vraie

valeur y .

- $\nabla L(\hat{y}, y)$ est le gradient de la fonction de perte par rapport aux paramètres $\tilde{w}$, sachant que $\hat{y} = s(\tilde{w} \cdot \tilde{x})$.
- $\tilde{w}^t$ représente la valeur des paramètres à l'itération t .
- η est le taux d'apprentissage.

Comme $\nabla L_{CE}(s(\tilde{w} \cdot \tilde{x}), y) = -(y - \hat{y})\tilde{x}$ alors l'équation ci-dessus devient :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta(s(\tilde{w}^t \cdot \tilde{x}) - y)\tilde{x}$$

Il existe plusieurs façons de calculer cette mise à jour :

- **Stochastique** : elle sélectionne un seul exemple aléatoire x à chaque itération. Puis elle ajuste les paramètres :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta(s(\tilde{w}^t \cdot \tilde{x}) - y)\tilde{x}$$

- **Par lots** : elle utilise tout le jeu d'entraînement pour calculer une seule mise à jour des paramètres.
- **Par mini-lots** : elle utilise un sous-ensemble X' de $\tilde{X}$, de taille n , pour calculer une seule mise à jour des paramètres :

$$\tilde{w}^{t+1} = \tilde{w}^t - \eta X'^T (s(X' \cdot (\tilde{w}^t)^T) - y)$$

où $(\tilde{w}^t)^T$ (resp. X'^T) est la transposée de $(\tilde{w}^t)$ (resp. X')

1.2.5.2 Régression logistique multinomiale

La régression logistique multinomiale permet de réaliser une classification avec plus de deux classes.

Étant donné un jeu de données étiqueté X , soit $\mathcal{K}$ l'ensemble des classes possibles et $K = |\mathcal{K}|$ leur nombre. Pour chaque observation x , l'étiquette correcte y est représentée par un vecteur one-hot⁵ de taille K . Si la classe correcte est c , alors $y_c = 1$ et $y_k = 0$ pour tout $k \neq c$.

Le classificateur prend en entrée une observation x et produit en sortie un vecteur $\hat{y} \in \mathbb{R}^K$. Chaque composante $\hat{y}_k$ représente la probabilité que l'observation appartienne à la classe k , avec :

$$\sum_{k=1}^K \hat{y}_k = 1.$$

Dans la régression logistique multinomiale, on associe à chaque classe k un vecteur de poids $w_k \in \mathbb{R}^f$ et un biais b_k . Ces paramètres peuvent être regroupés sous forme matricielle dans une matrice $W \in \mathbb{R}^{K \times f}$ et un vecteur de biais $b \in \mathbb{R}^K$, afin de simplifier les calculs.

Afin de simplifier les calculs, le biais b est intégré dans la matrice des poids W ce qui nous donne une nouvelle matrice $\tilde{W} \in \mathbb{R}^{K \times (f+1)}$.

Pour une observation x , le modèle calcule un score :

5. Un vecteur one-hot est un vecteur contenant un seul élément égal à 1 et tous les autres égaux à 0.

$$z = \tilde{W} \cdot \tilde{x}^T$$

La prédiction est ensuite donnée par :

$$\hat{y} = \text{softmax}(z)$$

où :

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{j=1}^K \exp(z_j)}, \dots, \frac{\exp(z_K)}{\sum_{j=1}^K \exp(z_j)} \right]$$

Chaque composante de $\hat{y}$ est comprise entre 0 et 1, et la somme des composantes du vecteur est égale à 1. Chaque composante k de $\hat{y}$ représente la probabilité que l'observation x appartienne à la classe k .

Pour un jeu de données X , on calcule alors les prédictions de tous les observations en une seule opération :

$$Z = \tilde{X} \cdot \tilde{W}^T$$

où $\tilde{W}^T$ est la transposée de $\tilde{W}$

Ensuite, on applique la fonction softmax ligne par ligne sur Z pour obtenir :

$$\hat{Y} = \text{softmax}(Z)$$

où $\hat{Y} \in \mathbb{R}^{m \times K}$, contenant pour chaque exemple une distribution de probabilité sur les classes. Comment les poids sont-ils appris ?

La fonction d'entropie croisée, utilisée pour mesurer l'écart entre la valeur prédite $\hat{y}$ par le modèle et la valeur réelle y , en régression logistique multinomiale pour une observation $\tilde{x}$ de $\tilde{X}$, est donnée par :

$$L_{CE}(\hat{y}, y) = - \sum_{i=1}^K y_i \log \hat{y}_i = - \log \hat{y}_k = - \log \left(\frac{\exp(\tilde{w}_k \cdot \tilde{x})}{\sum_{i=1}^K \exp(\tilde{w}_i \cdot \tilde{x})} \right)$$

où k est la classe correcte et $\tilde{w}_k$ représente les paramètres (poids et biais) associés à la classe k .

La fonction d'entropie croisée, utilisée pour mesurer l'écart entre les prédictions $\hat{Y}$ du modèle et les valeurs réelles Y , en régression logistique multinomiale sur l'ensemble du jeu d'entraînement $\tilde{X}$, est donnée par :

$$L_{CE}(\hat{Y}, Y) = - \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^K y_j^{(i)} \log \hat{y}_j^{(i)}$$

où :

— $\hat{Y} \in \mathbb{R}^{m \times K}$ est la matrice des prédictions ; chaque ligne i de $\hat{Y}$ correspond à une distri-

bution de probabilités sur les K classes pour l'exemple $\tilde{x}^{(i)}$.

- $Y \in \mathbb{R}^{m \times K}$ est la matrice des étiquettes réelles; chaque ligne i de Y est un vecteur one-hot représentant la classe réelle de l'exemple $\tilde{x}^{(i)}$.
- $y_j^{(i)}$ est la j -ème composante du vecteur cible $y^{(i)}$.
- $\hat{y}_j^{(i)}$ est la probabilité prédite par le modèle pour la classe j et l'exemple i .

On utilise ensuite un algorithme de descente de gradient pour minimiser cette fonction de perte.

Pour chaque classe k , le gradient de l'entropie croisée par rapport au vecteur de paramètres $\tilde{w}_k$ est donné par :

$$\nabla_{\tilde{w}_k} L_{CE}(\hat{Y}, Y) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_k^{(i)} - y_k^{(i)}) \tilde{x}^{(i)}$$

où :

- $\hat{y}_k^{(i)}$ est la probabilité prédite par le modèle pour la classe k et l'exemple i .
- $y_k^{(i)}$ est la valeur réelle (étiquette) pour la classe k et l'exemple i (issue d'un encodage one-hot).
- $\tilde{x}^{(i)}$ est le vecteur de caractéristiques de l'exemple i , éventuellement augmenté pour inclure le biais.

Les paramètres sont ensuite mis à jour selon la règle de descente de gradient :

$$\tilde{w}_k^{(t+1)} = \tilde{w}_k^{(t)} - \eta \frac{1}{m} \sum_{i=1}^m (\hat{y}_k^{(i)} - y_k^{(i)}) \tilde{x}^{(i)}$$

où :

- $\tilde{w}_k^{(t)}$ est le vecteur de paramètres de la classe k à l'itération t .
- $\tilde{w}_k^{(t+1)}$ est le vecteur de paramètres mis à jour après l'itération t .
- η est le taux d'apprentissage.

1.2.6 Réseaux de neurones récurrents.

Les réseaux de neurones sont une approche d'apprentissage automatique inspirée du fonctionnement des neurones biologiques du cerveau humain. Ils apprennent à partir de données et peuvent être utilisés pour des tâches de classification, de régression ou de prédiction. Ils sont particulièrement adaptés à la modélisation de relations non linéaires complexes.

Un réseau de neurones artificiel est généralement composé d'une couche d'entrée, d'une ou plusieurs couches cachées et d'une couche de sortie. La profondeur d'un réseau correspond au nombre de couches de neurones qu'il contient; la couche d'entrée n'est généralement pas comptée dans ce nombre. Chaque couche est composée d'un certain nombre d'unités appelées neurones artificiels. Le nombre de neurones d'une couche correspond à la taille de cette couche. La taille de la couche d'entrée est égale à la dimension du vecteur d'entrée x .

Dans un réseau de neurones classique entièrement connecté, les neurones d'une même couche ne sont généralement pas connectés entre eux. Les connexions se font entre couches successives : chaque neurone d'une couche est relié à tous les neurones de la couche suivante.

Chaque neurone d'une couche cachée ou de sortie reçoit en entrée un vecteur de valeurs

réelles x , associé à un vecteur de poids w , ainsi qu'un biais b , généralement un scalaire. Le neurone calcule d'abord une combinaison linéaire :

$$z = w \cdot x + b$$

Puis une fonction d'activation non linéaire f est appliquée afin d'obtenir la sortie du neurone :

$$y = f(z)$$

Cette fonction d'activation peut être, par exemple, la fonction sigmoïde, la tangente hyperbolique ($\tanh$) ou la fonction ReLU (Rectified Linear Unit). La sortie d'une couche est donc un vecteur contenant les sorties de tous ses neurones.

Lorsqu'une entrée x est fournie au réseau, l'information se propage de la couche d'entrée vers les couches cachées puis vers la couche de sortie. Afin de simplifier les calculs, les poids des neurones d'une même couche sont regroupés dans une matrice de poids, et les biais dans un vecteur de biais. Ces paramètres sont initialisés aléatoirement puis ajustés pendant l'entraînement afin de minimiser l'erreur du modèle.

Un réseau de neurones récurrent est tout réseau de neurones qui contient un cycle dans ses connexions internes ce qui signifie que la valeur d'une certaine unité neuronale dépend directement ou indirectement de ses propres sorties précédentes comme entrée.[4]. Grâce à cette propriété, les RNN sont capables de conserver une information sur le contexte précédent. Ils sont donc particulièrement adaptés au traitement de données séquentielles telles que les séries temporelles, le texte, la parole ou les séquences biologiques.

Pour comprendre le fonctionnement des RNN, considérons le cas d'un RNN simple, c'est-à-dire un réseau récurrent qui associe une séquence d'entrée à une séquence de sortie de même longueur. Les éléments de la séquence sont traités un à un, dans l'ordre.

La figure ci-dessous illustre la structure de ce réseau.

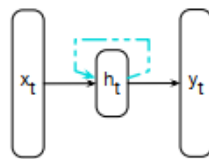


FIGURE 1.2 – Structure de la RNN simple

L'indice t représente le temps ou la position dans la séquence.

À chaque instant t :

- $x^{(t)}$ représente l'entrée du réseau ;
- $h^{(t)}$ représente la couche cachée du réseau ;
- $y^{(t)}$ représente la sortie du réseau.

La couche cachée $h^{(t)}$ dépend à la fois de l'entrée actuelle $x^{(t)}$ et de la couche cachée précédente $h^{(t-1)}$. Ainsi, la couche cachée $h^{(t)}$ influence donc directement la valeur de la sortie $y^{(t)}$ et indirectement les valeurs des couches cachées futures et sorties futures .

Le réseau récurrent peut être représenté sous deux formes :

- une représentation compacte avec une boucle récurrente ;
- une représentation déroulée dans le temps, où chaque instant de temps est représenté explicitement.

Le déroulement du réseau est particulièrement utile pour appliquer l'algorithme de rétropropagation à travers le temps (Backpropagation Through Time ou BPTT), utilisé pour calculer les gradients et mettre à jour les poids. Ces deux représentations sont illustrées dans la figure ci-dessous.

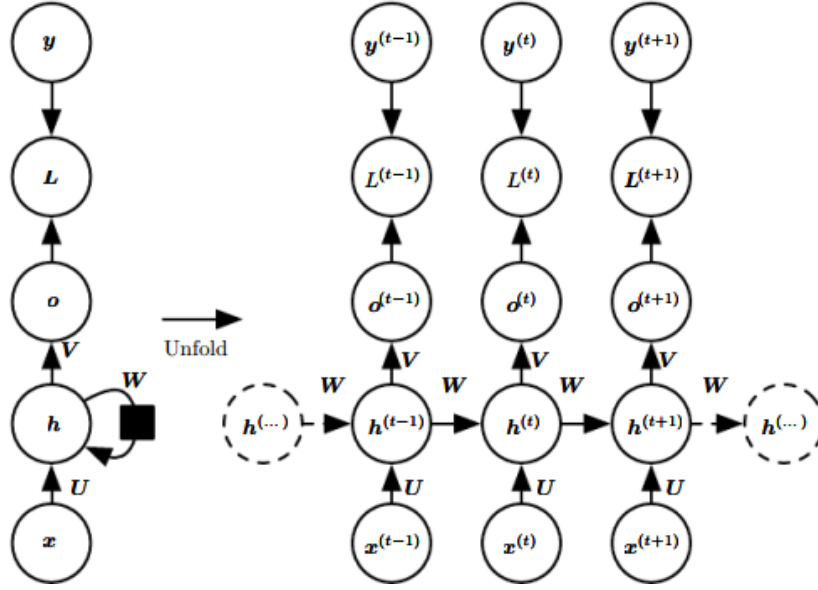


FIGURE 1.3 – La figure à gauche est une représentation compacte de la RNN simple et celle à droite est sa représentation déroulée dans le temps

D'après la figure ci-dessus, dans un RNN simple, le passage d'une séquence d'entrées à une séquence de sorties se fait de la manière suivante.

La matrice U relie l'entrée à la couche cachée, la matrice W relie l'état caché précédent à l'état caché courant, et la matrice V relie la couche cachée à la couche de sortie. Les mêmes matrices de poids sont utilisées à chaque instant de temps. Le calcul commence par l'initialisation de l'état caché initial, souvent :

$$h^{(0)} = 0$$

Ensuite, pour chaque instant t allant de 1 à T , où T est la longueur de la séquence, les calculs suivants sont effectués :

$$a^{(t)} = b + W \cdot h^{(t-1)} + U \cdot x^{(t)}$$

$$h^{(t)} = f(a^{(t)})$$

$$o^{(t)} = V \cdot h^{(t)} + c$$

$$y^{(t)} = g(o^{(t)})$$

où :

- f est la fonction d'activation non lineaire de la couche cachée.

- g est la fonction d'activation non lineaire de la couche de sortie.
- b est le vecteur de biais de la couche cachée.
- c est le vecteur de biais de la couche de sortie.

La séquence y est composé des $y^{(t)}$ pour t allant de 1 jusqu'à T .

L'objectif de l'entraînement est d'apprendre les matrices de poids W , U et V ainsi que les biais b et c . Pour cela, on utilise :

- un jeu de données d'entraînement ;
- une fonction de perte permettant de mesurer l'écart entre les prédictions du modèle et les valeurs réelles.

L'entraînement d'un réseau de neurones récurrent (RNN) se déroule généralement en deux étapes : la propagation avant et la rétropropagation à travers le temps.

La première étape, appelée propagation avant, consiste à faire passer chaque séquence d'entrée dans le réseau afin de calculer les sorties prédites. À chaque instant t , une perte locale $L^{(t)}$ est calculée en comparant la sortie prédite à la sortie réelle. La perte totale associée à une séquence est ensuite obtenue en additionnant les pertes sur tous les instants :

$$L = \sum_{t=1}^T L^{(t)}$$

où T désigne la longueur de la séquence.

La deuxième étape consiste à calculer les gradients de la perte totale par rapport aux paramètres du réseau (W , U , V , b et c). Ces gradients indiquent comment modifier les paramètres afin de réduire l'erreur du modèle.

Afin de faciliter le calcul des gradients, le réseau est déroulé dans le temps pour former un graphe computationnel. Cette représentation permet d'appliquer l'algorithme de rétropropagation à travers le temps.

Dans le graphe computationnel déroulé, les nœuds incluent les paramètres du réseau (W , U , V , b et c) ainsi que les variables dépendant du temps : $x^{(t)}$, $h^{(t)}$, $o^{(t)}$ et $L^{(t)}$.

Le calcul des gradients commence par les nœuds les plus proches de la fonction de perte. On calcule d'abord le gradient de la perte totale L par rapport à chaque perte locale $L^{(t)}$:

$$\frac{\partial L}{\partial L^{(t)}} = 1$$

Ensuite, on calcule le gradient de L par rapport à la sortie $o^{(t)}$ pour chaque instant t :

$$\nabla_{o^{(t)}} L = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o^{(t)}}$$

Au dernier instant T , l'état caché $h^{(T)}$ influence uniquement la sortie $o^{(T)}$. Son gradient est

donc donné par :

$$\nabla_{h^{(T)}} L = V^\top \nabla_{o^{(T)}} L$$

Pour les instants précédents, chaque état caché $h^{(t)}$ influence à la fois la sortie courante $o^{(t)}$ et l'état caché suivant $h^{(t+1)}$. Le gradient doit donc être propagé à travers ces deux dépendances.

On itère ainsi de $t = T - 1$ jusqu'à $t = 1$ afin de calculer récursivement le gradient par rapport à chaque état caché :

$$\nabla_{h^{(t)}} L = \left(\frac{\partial h^{(t+1)}}{\partial h^{(t)}} \right)^\top \nabla_{h^{(t+1)}} L + \left(\frac{\partial o^{(t)}}{\partial h^{(t)}} \right)^\top \nabla_{o^{(t)}} L$$

Une fois les gradients calculés sur les nœuds internes du graphe computationnel, on peut calculer les gradients par rapport aux paramètres du réseau.

Le gradient par rapport au biais de sortie c est donné par :

$$\nabla_c L = \sum_t \left(\frac{\partial o^{(t)}}{\partial c} \right)^\top \nabla_{o^{(t)}} L$$

Le gradient par rapport au biais caché b est :

$$\nabla_b L = \sum_t \left(\frac{\partial h^{(t)}}{\partial b} \right)^\top \nabla_{h^{(t)}} L$$

Le gradient par rapport à la matrice de poids V est :

$$\nabla_V L = \sum_t (\nabla_{o^{(t)}} L) (h^{(t)})^\top$$

Le gradient par rapport à la matrice récurrente W est :

$$\nabla_W L = \sum_t \sum_i \frac{\partial L}{\partial h_i^{(t)}} \nabla_{W^{(t)}} h_i^{(t)}$$

Le gradient par rapport à la matrice d'entrée U est :

$$\nabla_U L = \sum_t \sum_i \frac{\partial L}{\partial h_i^{(t)}} \nabla_{U^{(t)}} h_i^{(t)}$$

Les gradients calculés sont ensuite utilisés pour mettre à jour les paramètres du réseau à l'aide d'un algorithme d'optimisation basé sur le gradient, comme la descente de gradient.

À l'itération $i + 1$, les paramètres sont mis à jour selon les équations suivantes :

$$\begin{aligned} c^{(i+1)} &= c^{(i)} - \eta \nabla_c L \\ b^{(i+1)} &= b^{(i)} - \eta \nabla_b L \end{aligned}$$

$$V^{(i+1)} = V^{(i)} - \eta \nabla_V L$$

$$W^{(i+1)} = W^{(i)} - \eta \nabla_W L$$

$$U^{(i+1)} = U^{(i)} - \eta \nabla_U L$$

où η désigne le taux d'apprentissage.

Chapitre 2

Byte Pair Encoding

Le Byte Pair Encoding (BPE) est une technique de compression de données consistant à fusionner itérativement les paires d'octets ou de caractères consécutifs les plus fréquentes d'un texte ou d'une séquence de données en un nouveau symbole unique. Le processus est répété jusqu'à atteindre un nombre prédéfini de fusions ou jusqu'à ce qu'il ne reste plus de paires fréquentes.

Cette méthode a été décrite pour la première fois en 1994 par Philip Gage. Elle est ensuite devenue l'une des techniques de tokenisation en sous-mots les plus utilisées en traitement automatique du langage naturel (TALN).

2.1 Présentation.

L'idée de l'algorithme est de représenter les paires les plus fréquentes par des marqueurs dédiés. Ce faisant, il identifie les séquences d'éléments (caractères) les plus fréquentes. Ensuite, en utilisant ces informations, il peut encoder des textes.

2.1.1 Logique principale.

La logique de l'algorithme est illustrée par les étapes suivantes :

1. Construire un vocabulaire à partir d'un corpus de mots.
2. Identifier la paire la plus fréquente : compter les paires de symboles et sélectionner celle ayant la fréquence la plus élevée.
3. Produire un nouvel élément de vocabulaire pour la paire la plus fréquente : fusionner cette paire en un nouveau symbole et mettre à jour le corpus afin de refléter ce nouvel élément.
4. Répéter les étapes précédentes jusqu'à atteindre un nombre cible de fusions.
5. Enregistrer la table des fusions.
6. Encoder une chaîne de caractères à l'aide de la table des fusions.

2.1.2 Illustration à travers un exemple.

```
b := BytePairEncoder new.  
b fromText: 'FRED FED TED BREAD, AND TED FED FRED BREAD'.
```

Inspection du corpus.

Tout d'abord, une fois le texte d'entrée tokenisé, on obtient le vocabulaire suivant.

```
corpus  
#('F' 'E' 'D' ' ') 2  
#('B' 'R' 'E' 'A' 'D' ' ') 2  
#('A' 'N' 'D' ' ') 1  
#('T' 'E' 'D' ' ') 2  
#('F' 'R' 'E' 'D' ' ') 2
```

Lorsque l'on observe le système, on obtient les éléments suivants :

- un vocabulaire : 'T' 'A' 'F' 'D' 'B' ' ' 'R' 'N' 'E'
- un corpus : comme indiqué ci-dessus
- une liste de paires consécutives avec leurs fréquences
- une liste de fusions vide

```
voc  
'T' 'A' 'F' 'D' 'B' ' ' 'R' 'N' 'E'
```

```
corpus  
#('F' 'E' 'D' ' ') 2  
#('B' 'R' 'E' 'A' 'D' ' ') 2  
#('A' 'N' 'D' ' ') 1  
#('T' 'E' 'D' ' ') 2  
#('F' 'R' 'E' 'D' ' ') 2
```

```
pairs  
#('N' 'D') 1  
#('B' 'R') 2  
#('A' 'N') 1  
#('A' 'D') 2  
#('F' 'E') 2  
#('D' ' ') 9  
#('R' 'E') 4  
#('E' 'A') 2  
#('T' 'E') 2  
#('E' 'D') 6  
#('F' 'R') 2
```

```
merges
```

On observe que la paire la plus fréquente est $\#('A' 'B')$. Elle sera donc celle qui sera fusionnée.

Itération 1.

Maintenant, examinons la situation avec la première fusion.

b mergeOneStep

On observe que la paire la plus fréquente $\#('D' '_')$ a été fusionnée. Cela signifie que :

- Elle a été ajoutée au vocabulaire : ici, nous obtenons par exemple $'D_'$ comme nouvel élément. À noter que si les caractères étaient encodés par des nombres, les éléments fusionnés seraient simplement représentés par un nouvel identifiant numérique.
- Elle a été ajoutée à la liste des fusions.
- Le corpus a été mis à jour afin de refléter la présence du nouvel élément du vocabulaire. Par exemple, $\#('F' 'E' 'D' '_')$ est désormais encodé comme $\#('F' 'E' 'D_')$.
- Toutes les paires ont été recalculées à partir de la version mise à jour du corpus. Ainsi, la paire la plus fréquente devient maintenant $\#('E' 'D_')$.

voc

$'F' 'R' 'T' 'B' 'N' 'D_ ' '_ 'E' 'D' 'A'$

corpus

$\#('F' 'E' 'D_') 2$

$\#('A' 'N' 'D_') 1$

$\#('B' 'R' 'E' 'A' 'D_') 2$

$\#('T' 'E' 'D_') 2$

$\#('F' 'R' 'E' 'D_') 2$

pairs

$\#('E' 'D_') 6$

$\#('B' 'R') 2$

$\#('A' 'N') 1$

$\#('N' 'D_') 1$

$\#('F' 'E') 2$

$\#('E' 'A') 2$

$\#('R' 'E') 4$

$\#('T' 'E') 2$

$\#('F' 'R') 2$

$\#('A' 'D_') 2$

merges

$\#('D' '_') \rightarrow 'D_'$

Itération 2.

L'algorithme répète le même processus. Ainsi, la fusion $\#('E' 'D_') \rightarrow 'ED_'$ est ajoutée à la liste des fusions. Le vocabulaire, le corpus et les paires sont ensuite mis à jour.

```

voc
  'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' ' ' 'E' 'D' 'A'
corpus
  #('T' 'ED_') 2
  #('F' 'R' 'ED_') 2
  #('A' 'N' 'D_') 1
  #('B' 'R' 'E' 'A' 'D_') 2
  #('F' 'ED_') 2
pairs
  #('T' 'ED_') 2
  #('B' 'R') 2
  #('A' 'N') 1
  #('N' 'D_') 1
  #('R' 'E') 2
  #('E' 'A') 2
  #('F' 'R') 2
  #('A' 'D_') 2
  #('R' 'ED_') 2
  #('F' 'ED_') 2
merges
  #('D' ' ') -> 'D_'
  #('E' 'D_') -> 'ED_'

```

On observe maintenant qu'il existe plusieurs paires ayant la fréquence la plus élevée. L'algorithme en sélectionnera une parmi elles.

Itération 3.

Voici l'itération suivante. La fusion $\#('T' 'ED_') \rightarrow 'TED_'$ a été ajoutée à la liste des fusions. Il est à noter qu'il s'agit de la première paire apparaissant dans le journal présenté ici, mais cela est simplement dû au fait que la logique d'affichage utilise la même implémentation de type *bag* ainsi que la même logique de tri que celle utilisée par l'algorithme lorsqu'il recherche l'association la plus fréquente. Cela n'est pas lié à l'algorithme lui-même. Une autre paire ayant la même fréquence aurait pu être sélectionnée.

```

voc
  'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' ' ' 'E' 'TED_' 'D' 'A'
corpus
  #('F' 'R' 'ED_') 2
  #('A' 'N' 'D_') 1
  #('B' 'R' 'E' 'A' 'D_') 2
  #('TED_') 2

```

```

#('F' 'ED_') 2
pairs
#('B' 'R') 2
#('A' 'N') 1
#('N' 'D_') 1
#('R' 'E') 2
#('E' 'A') 2
#('F' 'R') 2
#('A' 'D_') 2
#('R' 'ED_') 2
#('F' 'ED_') 2
merges
#('D' ' _')->'D_'
#('E' 'D_')->'ED_'
#('T' 'ED_')->'TED_'

```

Itération 4. Voici une autre itération au cours de laquelle la paire #('B' 'R') a été sélectionnée.

```

voc
'BR' 'F' 'R' 'ED_' 'T' 'B' 'N' 'D_' ' _' 'E' 'TED_' 'D' 'A'
corpus
#('F' 'R' 'ED_') 2
#('F' 'ED_') 2
#('A' 'N' 'D_') 1
#('TED_') 2
#('BR' 'E' 'A' 'D_') 2
pairs
#('BR' 'E') 2
#('A' 'N') 1
#('N' 'D_') 1
#('E' 'A') 2
#('F' 'R') 2
#('F' 'ED_') 2
#('R' 'ED_') 2
#('A' 'D_') 2
merges
#('D' ' _')->'D_'
#('E' 'D_')->'ED_'
#('T' 'ED_')->'TED_'
#('B' 'R')->'BR'

```

Jusqu'à ce qu'aucune fusion supplémentaire ne soit possible.

L'algorithme continue comme expliqué jusqu'à ce qu'il n'y ait plus de paires à fusionner, c'est-à-dire lorsque la fréquence des paires est égale à un. On obtient la situation suivante.

```
oc
  'BR' 'BREA' 'R' 'TED_' 'ED_' 'E' 'N' 'FR' 'A' 'BRE' ' ' 'F' 'D_' 'BREAD_' 'D' 'FRED_'
corpus
  #('FRED_') 2
  #('FED_' ) 2
  #('BREAD_' ) 2
  #('A' 'N' 'D_' ) 1
  #('TED_' ) 2
pairs
  #('A' 'N') 1
  #('N' 'D_' ) 1
merges
  #('D' ' ') -> 'D_'
  #('E' 'D_' ) -> 'ED_'
  #('T' 'ED_' ) -> 'TED_'
  #('B' 'R') -> 'BR'
  #('BR' 'E') -> 'BRE'
  #('BRE' 'A') -> 'BREA'
  #('F' 'R') -> 'FR'
  #('BREA' 'D_' ) -> 'BREAD_'
  #('F' 'ED_' ) -> 'FED_'
  #('FR' 'ED_' ) -> 'FRED_'
```

Utilisation des fusions.

Nous pouvons maintenant encoder un texte. La logique de cette étape est la suivante :

- tokeniser le texte ;
- parcourir la liste des fusions dans l'ordre et appliquer chaque fusion l'une après l'autre sur le résultat de l'application précédente ;
- une fois arrivé à la fin de la liste des fusions, on obtient le texte encodé.

Imaginons que nous voulions encoder 'FRED EATS BREAD'. Lors de la phase de tokenisation, nous obtenons la liste suivante :

```
'F' 'R' 'E' 'D' ' ' 'E' 'A' 'T' 'S' ' ' 'B' 'R' 'E' 'A' 'D_'
```

À noter que l'on pourrait également travailler au niveau des mots et utiliser une structure de type Bag afin de factoriser le traitement pour des mots similaires.

Ensuite, nous demandons à l'algorithme d'encoder le texte. On obtient la liste d'éléments suivante : 'FRED_' 'E' 'A' 'T' 'S' ' ' 'BREAD'

```
self encode: 'FRED EATS BREAD'
> an OrderedCollection('FRED_' 'E' 'A' 'T' 'S' '_' 'BREAD_')
```

Ce que l'on observe, c'est que les mots fréquents ont été substitués. Ainsi, si on l'examine sous une forme numérique, on obtient :

```
'ABRC' asArray collect: [ :each | each digitValue -9 ]
> #(1 2 18 3)

'FRED EATS BREAD' asArray collect: [ :each | each digitValue -9 ]
> #(6 18 5 4 -10 5 1 20 19 -10 2 18 5 1 4)

'EATS_' asArray collect: [ :each | each digitValue -9 ]
> #(5 1 20 19 -10)
```

Par conséquent, le texte passe de 15 à 7 éléments (car 'FRED_' et 'BREAD_' comptent chacun comme un seul élément).

2.1.3 Observation sur l'algorithme.

L'exécution que nous avons présentée correspond à une implémentation naïve et simple. Elle reconstruit l'ensemble complet des paires à chaque itération. De plus, une seule paire est fusionnée à la fois.

Ce que montre cette exécution, c'est que le vocabulaire, le corpus et les paires doivent être mis à jour à chaque fusion.

Plusieurs paires pourraient être fusionnées au cours d'un même passage si l'algorithme prend en compte le fait que les paires fusionnées simultanément ne doivent ni commencer ni se terminer par un élément similaire.

Lorsque l'on dispose des paires suivantes :

```
#('N' 'D') 1
#('B' 'R') 2
#('A' 'N') 1
#('A' 'D') 2
#('F' 'E') 2
#('D' '_' ) 9
#('R' 'E') 4
#('E' 'A') 2
#('T' 'E') 2
#('E' 'D') 6
#('F' 'R') 2
```

Lors de la fusion de #('D' ") 9, l'algorithme ne devrait pas fusionner les paires se terminant par 'D' ou commençant par '_', mais il pourrait en fusionner d'autres, comme 'R' 'E'. Or, la fusion simultanée de plusieurs paires conduit à des encodages différents.

2.2 Implémentation.

L'objectif cette section est d'implémenter l'algorithme Byte Pair Encoding (BPE) en Pharo. Le fonctionnement de BPE est décrit la section 1.3.1.1.

2.2.1 Algorithme pseudo-code.

Algorithm 1: Train

Input: Text T , merge count N
Output: Merges dictionary M , vocabulary V
 $V \leftarrow \text{initializeVocabulary}(T)$
 $\text{corpus} \leftarrow \text{computeWordsFrequency}(T)$
 $M \leftarrow \{\}$
 $i \leftarrow N - \text{size}(V)$
if $i \leq 0$ **then**
 return M, V
else
 $j \leftarrow 1$
 while $i > 0$ **do**
 $\text{pairs} \leftarrow \text{computePairs}(\text{corpus})$
 $\text{bestPair} \leftarrow \text{max}(\text{pairs})$
 $\text{newSymbol} \leftarrow \text{bestPair}[1] + \text{bestPair}[2]$
 $M[\text{newSymbol}] \leftarrow j$
 $V \leftarrow \text{add}(V, \text{newSymbol})$
 $\text{corpus} \leftarrow \text{updateCorpus}(\text{corpus}, \text{bestPair}, \text{newSymbol})$
 $j = j + 1$
 $i \leftarrow i - 1$
 return M, V

L'algorithme 1 permet l'entraînement du BPE sur un corpus de texte. Il prend en entrée un texte ainsi que le nombre de fusions à effectuer.

Algorithm 2: computeWordsFrequency

Input: Text T **Output:** Corpus $corpus$ $corpus \leftarrow \{\}$ $word \leftarrow []$ $i \leftarrow 1$ $j \leftarrow 1$ **while** $i \leq \text{length}(T)$ **do** **if** $T[i] \neq ""$ **then** $word[j] \leftarrow T[i]$ $j \leftarrow j + 1$ **if** $T[i] = ""$ **and** $\text{length}(word) \neq 0$ **then** $word[j] \leftarrow _$ **if** $word \in corpus$ **then** $corpus[word] \leftarrow corpus[word] + 1$ **else** $corpus[word] \leftarrow 1$ $word \leftarrow []$ $j \leftarrow 1$ $i \leftarrow i + 1$ **return** $corpus$

L'algorithme 2 parcourt le texte d'entrée afin d'extraire les mots, d'ajouter le symbol spécial "_" à la fin de chaque mot et de calculer leur fréquence d'apparition. Elle retourne un dictionnaire associant chaque mot à son nombre d'occurrences dans le corpus.

Algorithm 3: computePairs

Input: Corpus $corpus$ **Output:** Pairs $pairs$ $pairs \leftarrow \{\}$ **foreach** $(key, value) \in corpus$ **do** $i \leftarrow 1$ **while** $i < \text{length}(key)$ **do** $pair \leftarrow [key[i], key[i + 1]]$ **if** $pair \in pairs$ **then** $pairs[pair] \leftarrow pairs[pair] + value$ **else** $pairs[pair] \leftarrow value$ $i \leftarrow i + 1$ **return** $pairs$

Cet algorithme 3 construit un dictionnaire des fréquences des paires de symboles adjacents présentes dans un corpus.

Algorithm 4: updateCorpus

Input: Corpus *corpus*, most frequent pair *pair*, new symbol *symbol*

Output: Updated corpus *corpus1*

corpus1 $\leftarrow \{\}$

foreach $(key, value) \in corpus$ **do**

i $\leftarrow 1$

word $\leftarrow []$

j $\leftarrow 1$

while *i* < *length*(*key*) **do**

if *key*[*i*] = *pair*[1] **and** *key*[*i* + 1] = *pair*[2] **then**

word[*j*] $\leftarrow symbol$

i $\leftarrow i + 2$

else

word[*j*] $\leftarrow key[i]$

i $\leftarrow i + 1$

j $\leftarrow j + 1$

corpus1[*word*] $\leftarrow value$

return *corpus1*

L'algorithme 4 met à jour le corpus en remplaçant les occurrences de la paire de symboles la plus fréquente par un nouveau symbole. Chaque mot du corpus est parcouru, et les paires correspondantes sont fusionnées afin de produire un corpus mis à jour.

Algorithm 5: Tokenize

Input: Text *text*, Merges dictionary *merges*

Output: Tokenized text *tokens*

corpus $\leftarrow$ split(*text*)

a = True

tokens $\leftarrow$ []

while *a* = True **do**

pair $\leftarrow$ findPairToMerge(*corpus*, *merges*)

if *pair* = null **then**

a $\leftarrow$ False

else

corpus $\leftarrow$ updateCorpusWithNewSymbol(*corpus*, *pair*)

i = 1

k = 1

while *i* $\leq$ length(*corpus*) **do**

j $\leftarrow$ 1

symbols $\leftarrow$ *corpus*[*i*]

while *j* $\leq$ length(*symbols*) **do**

tokens[*k*] $\leftarrow$ *symbols*[*j*]

k $\leftarrow$ *k* + 1

j $\leftarrow$ *j* + 1

i $\leftarrow$ *i* + 1

return *tokens*

L'algorithme 5 applique la tokenisation BPE à un texte donné en utilisant un ensemble de règles de fusion préalablement apprises. Il commence par découper le texte en unités de base, puis applique itérativement les fusions les plus pertinentes jusqu'à ce qu'aucune paire ne puisse être fusionnée. Enfin, il transforme le corpus obtenu en une séquence de tokens représentant le texte segmenté.

Algorithm 6: Split

Input: Text *text***Output:** Splited text *corpus**corpus* $\leftarrow []$ *tokens* $\leftarrow []$ *i* $\leftarrow 1$ *j* $\leftarrow 1$ *k* $\leftarrow 1$ **while** $i \leq \text{length}(\text{text})$ **do** **if** $\text{text}[i] \neq ""$ **then** *tokens*[*j*] $\leftarrow \text{text}[i]$ *j* $\leftarrow j + 1$ **if** $\text{text}[i] = ""$ **and** $\text{length}(\text{tokens}) \neq 0$ **then** *tokens*[*j*] $\leftarrow ""$ *corpus*[*k*] $\leftarrow \text{tokens}$ *tokens* $\leftarrow []$ *j* $\leftarrow 1$ *k* $\leftarrow k + 1$ *i* $\leftarrow i + 1$ **return** *corpus*

L'algorithme 6 découpe un texte en mots en parcourant les caractères un par un. Chaque mot est transformé en une séquence de symboles, à laquelle est ajouté un symbole spécial de fin de mot. Le résultat est un corpus où chaque mot est représenté comme une liste de caractères.

Algorithm 7: FindPairToMerge

Input: Corpus *corpus*, Merges dictionary *merge***Output:** Pair *pair**i* $\leftarrow 1$ *bestScore* $\leftarrow 0$ *pair* $\leftarrow \text{null}$ **while** $i \leq \text{length}(\text{corpus})$ **do** *tokens* $\leftarrow \text{corpus}[i]$ *j* $\leftarrow 1$ **while** $j < \text{length}(\text{tokens})$ **do** *symbol* $\leftarrow \text{tokens}[j] + \text{tokens}[j + 1]$ **if** $\text{symbol} \in \text{merge}$ **then** *score* $\leftarrow \text{merge}[\text{symbol}]$ **if** $\text{score} > \text{bestScore}$ **then** *pair* $\leftarrow [\text{tokens}[j], \text{tokens}[j + 1]]$ *j* $\leftarrow j + 1$ *i* $\leftarrow i + 1$ **return** *pair*

L'algorithme 7 parcourt le corpus afin d'identifier la paire de symboles adjacents la plus fréquente selon les règles de fusion disponibles. Pour chaque mot, il évalue les paires consécutives et récupère leur fréquence dans le dictionnaire des merges. La paire ayant le score le plus élevé est sélectionnée comme la meilleure paire à fusionner.

Algorithm 8: updateCorpusWithNewSymbol

Input: Corpus *corpus*, pair *pair*
Output: Corpus *corpus1*

```

i ← 1
corpus1 ← []
while i ≤ length(corpus) do
    tokens ← corpus[i]
    tokens1 ← []
    j ← 1
    k ← 1
    while j < length(tokens) do
        if tokens[j] = pair[1] and tokens[j + 1] = pair[2] then
            tokens1[k] ← pair[1] + pair[2]
            j ← j + 2
        else
            tokens1[k] ← tokens[j]
            j ← j + 1
        k ← k + 1
    corpus1[i] ← tokens1
    i ← i + 1
return corpus1

```

L'algorithme 8 met à jour le corpus en remplaçant chaque occurrence de la paire de symboles donnée par un nouveau symbole fusionné. Il parcourt chaque mot du corpus, détecte les occurrences de la paire cible, puis reconstruit une nouvelle représentation du mot avec les symboles fusionnés.

2.2.2 Implémentation en Pharo.

Nous allons maintenant présenter l'implémentation de l'algorithme de tokenisation BPE en Pharo. La version présentée dans ce chapitre est une version naïve, qui sera ensuite optimisée dans le chapitre suivant. Cette implémentation est disponible sur ce dépôt GitHub : BytePairEncoder GitHub Repository.

Création de classe.

Une classe a été créée afin de regrouper les variables et les méthodes nécessaires à l'implémentation du BPE. Elle permet de stocker les données du modèle, telles que le vocabulaire et les règles de fusion, ainsi que les opérations utilisées durant l'entraînement et la tokenisation.

Une classe `BytePairEncoder` a ainsi été créée avec les variables d'instance suivantes : `words`,

vocabulary, pairs et merges.

```
Class {  
  #name : 'BytePairEncoder',  
  #superclass : 'Object',  
  #instVars : [  
    'words',  
    'pairs',  
    'vocabulary',  
    'merges'  
  ],  
  #category : 'BytePairEncoder',  
  #package : 'BytePairEncoder'  
}
```

Structures de données utilisées.

- Une instance de la classe **Set** est utilisée pour représenter le vocabulaire, constitué des caractères initiaux du corpus ainsi que des symboles issus des fusions successives.
- Chaque mot du corpus est représenté sous forme de séquence ordonnée de caractères à l'aide d'une instance de la classe **OrderedCollection**, afin de préserver l'ordre des symboles.
- Une instance de la classe **Bag** est utilisée pour stocker les mots du corpus et leurs fréquences d'occurrence, chaque mot étant représenté comme une séquence de caractères.
- Une instance de la classe **Bag** est également utilisée pour comptabiliser les paires consécutives de caractères ou de sous-mots ainsi que leurs occurrences dans le corpus.
- Une instance de la classe **OrderedCollection** est utilisée pour conserver l'historique des fusions, en enregistrant chaque paire fusionnée ainsi que le symbole résultant.

Remarque : La classe **Bag** est utilisée car elle permet d'associer automatiquement les occurrences à chaque élément lors de leur insertion. Il suffit d'ajouter les éléments pour que leurs fréquences soient mises à jour : elle facilite le comptage des fréquences de chaque élément. De plus, elle fournit des opérations utiles comme la récupération des éléments les plus fréquents. L'accès aux éléments et à leurs occurrences est également simple et efficace.

Initialisation des variables d'instance.

La méthode `initialize` permet d'initialiser les variables d'instance : `words`, `vocabulary` et `merges`. La variable `pairs` n'est pas initialisée dans cette méthode, car elle est recalculée à chaque étape de l'entraînement.

```
{ #category : 'initialization' }  
BytePairEncoder >> initialize [
```

```
  super initialize.  
  words := Bag new.
```

```
"We use a bag so that we can have the occurrences at the world level."
```

```
vocabulary := Set new.  
merges := OrderedCollection new  
]
```

La structure de données **Bag** est utilisée pour **words** afin de stocker les mots du corpus avec leurs fréquences. Cela facilite le comptage des occurrences et permet d'éviter de répéter plusieurs fois les mêmes traitements sur des mots identiques.

vocabulary sert à stocker les symboles appris durant l'entraînement. Il sera ensuite utilisé par le modèle de langage. Au début, il est initialisé comme une instance de **Set**, puis les caractères présents dans le corpus y sont ajoutés, y compris le symbole spécial "_" indiquant la fin d'un mot. Ensuite, à chaque étape de l'entraînement, le nouveau symbole obtenu par la fusion de la paire la plus fréquente est ajouté au vocabulaire.

merges est une instance de **OrderedCollection**, utilisée pour stocker les règles de fusion apprises durant l'entraînement. L'ordre des fusions est important, car ces règles devront être réappliquées dans le même ordre lors de la tokenisation de nouveaux textes.

Prétraitement du texte.

Le texte est d'abord découpé en mots, puis chaque mot est transformé en une collection ordonnée de caractères grâce à la méthode **textToWords** :

```
{ #category : 'compute' }  
BytePairEncoder >> textToWords: aText [  
    "Split the text into words. Then turn a word as a collection of strings."  
  
    | rawWords |  
    rawWords := aText substrings: {  
        Character space.  
        Character cr.  
        Character tab . $. . $? . $! . $: . $; }.  
    ^ (rawWords collect: [ :each |  
        | col |  
        col := OrderedCollection new: each size + 1.  
        each do: [ :aChar | col add: aChar asString ].  
        col add: '_'.  
        col ] )  
]
```

Cette méthode sera ensuite utilisée par **prepareWordsFromText** : pour ajouter les mots transformés dans la variable **words** :

```
{ #category : 'compute' }
```

```
BytePairEncoder >> prepareWordsFromText: aText [
  "Split the text into words. Then build the corpus of words by representing a word as a

  words addAll: (self textToWords: aText)
]
```

Ainsi, chaque mot est représenté comme une collection ordonnée de chaînes de caractères. Par exemple, le mot "low" devient #('l' 'o' 'w' '_'). Cette représentation facilite les opérations de fusion durant l'entraînement du BPE.

Construction du vocabulaire.

Le vocabulaire est construit grâce à la méthode `computeVocabulary` en parcourant chaque élément de `words` :

```
{ #category : 'vocabulary' }
BytePairEncoder >> computeVocabulary [
  "we do not add characters but string because in the merge phase we will have sequences

  words do: [ :each |
    each do: [ :aChar |
      self addToVocabulary: aChar asString ]].
]
```

Chaque caractère est converti en chaîne de caractères avant son ajout au vocabulaire, car les opérations de fusion en Pharo s'effectuent sur des chaînes de caractères.

Calcul de la fréquence de chaque paire existante du corpus.

Les paires de symboles adjacents sont calculées grâce à la méthode `computePairs` en parcourant chaque élément de `words` :

```
BytePairEncoder >> computePairs [

  pairs := Bag new.
  words associationsDo: [ :association |
    | word |
    word := association key.
    word overlappingPairsWithIndexDo: [ :a :b :index |
      pairs add: { a . b } withOccurrences: association value ]
  ]
]
```

La méthode `overlappingPairsWithIndexDo:` parcourt une collection par paires d'éléments adjacents et exécute un bloc pour chaque paire et l'indice du premier élément de la paire.

Initialisation du pipeline BPE.

Les méthodes `prepareWordsFromText:`, `computeVocabulary` et `computePairs` sont regroupées dans `fromText:` afin de préparer le texte avant l'entraînement, d'initialiser le vocabulaire et de calculer les fréquences initiales des paires. Cette méthode reçoit un texte en entrée.

```
{ #category : 'api' }
BytePairEncoder >> fromText: aString [

    self prepareWordsFromText: aString.
    self computeVocabulary.
    self computePairs.
]
```

Entraînement du BPE.

Après cette phase de préparation, l'entraînement du BPE consiste à :

- déterminer la paire la plus fréquente ;
- fusionner cette paire ;
- ajouter le nouveau symbole au vocabulaire ;
- enregistrer la règle de fusion ;
- mettre à jour le corpus ;
- recalculer les fréquences des paires.

Ces opérations sont regroupées dans la méthode `mergeOneStep :`

```
{ #category : 'merges' }
BytePairEncoder >> mergeOneStep [

    | newVocabularyItem pair |
    pair := pairs sortedCounts first value.
    newVocabularyItem := pair first, pair second.
    self addToVocabulary: newVocabularyItem.
    self addToMergesPair: pair mergedInto: newVocabularyItem.
    self updateCorpusFrom: pair to: newVocabularyItem.
    self computePairs.
]
```

Cette méthode est exécutée plusieurs fois jusqu'à atteindre la taille de vocabulaire souhaitée ou jusqu'à ce qu'il n'existe plus de paires à fusionner.

La mise à jour du corpus est réalisée grâce à la méthode `updateCorpusFrom:to: :`

```
{ #category : 'merges' }
BytePairEncoder >> updateCorpusFrom: pair to: newVocabularyItem [

    | bag |
```

```

bag := Bag new.
words keys copy do:
[ :word |
bag
add: (word copyReplaceAll: pair with: {newVocabularyItem})
withOccurrences: (words occurrencesOf: word)
].
words := bag
]

```

Cette méthode remplace toutes les occurrences de la paire donnée par le nouveau symbole fusionné dans chaque mot du corpus.

Après chaque mise à jour du corpus, les fréquences des paires sont recalculées en exécutant de nouveau `computePairs`.

Application des règles de fusion apprises durant l'entraînement.

Une fois l'entraînement terminé, les règles de fusion stockées dans `merges` peuvent être appliquées à un nouveau texte grâce à la méthode `encode: :`

```

{ #category : 'api' }
BytePairEncoder >> encode: aString [
    "this encoding does not considering overlapping merges.

    if we have
    a,b -> 1
    b,c -> 2
    and a string
    abc
    it will produce
    1c
    and not 12
    "

    | wrds |
    wrds := (self textToWords: aString) flatCollect: [ :each | each ].
    wrds := wrds asOrderedCollection.

    merges do: [ :aMerge |
        | key first second rempl i |
        key := aMerge key.
        first := key first.
        second := key second.
        rempl := aMerge value.

```

```

i := 1.
[ i < wrds size ] whileTrue: [
  ((wrds at: i) = first)
and: [ (wrds at: i + 1) = second ])
ifTrue: [ wrds removeAt: i; at: i put: rempl.
i := i + 1].
i := i + 1 ].
].
^ wrds
]

```

Avant d'appliquer les règles de fusion, le texte est prétraité avec `textToWords:` afin d'être transformé en collections ordonnées de symboles. La méthode `flatCollect:` est ensuite utilisée pour aplatir ces collections et obtenir une unique séquence de symboles représentant le texte.

La méthode `flatCollect:` n'a pas été utilisée durant l'entraînement afin d'éviter la création de paires composées de symboles appartenant à des mots différents. Cela permet de préserver les frontières entre les mots et d'empêcher la fusion de symboles provenant de mots distincts.

Chapitre 3

WordPiece

3.1 Présentation de WordPiece.

WordPiece est un algorithme de tokenisation en sous-mots. Il découpe les mots en caractères ou en sous-mots selon les vocabulaires obtenus lors de l'entraînement de WordPiece à partir d'un corpus. Cet algorithme a été développé initialement par Google et utilisé pour le pré-entraînement du modèle BERT. L'algorithme présenté ici est inspiré des explications proposées dans le cours de Hugging Face.

3.1.1 Comprendre WordPiece.

L'idée principale de WordPiece est de sélectionner, à chaque itération, la paire de symboles ayant le score le plus élevé. Ce score est calculé à partir de la formule suivante :

$$score = \frac{\text{fréquence de la paire}}{\text{fréquence du premier élément de la paire} * \text{fréquence du second élément de la paire}}$$

La paire sélectionnée est ensuite fusionnée pour former un nouveau symbole ajouté au vocabulaire. Le vocabulaire obtenu après les itérations est ensuite utilisé pour encoder du texte.

3.1.2 Logique de fonctionnement de WordPiece.

Le fonctionnement de WordPiece peut être résumé par les étapes suivantes :

1. Prétraiter le texte : chaque mot est transformé en une collection ordonnée de caractères. Les caractères situés au milieu d'un mot sont préfixés par le symbole spécial "##".
2. Initialiser le vocabulaire : le vocabulaire initial est constitué des caractères présents dans le corpus. Les caractères apparaissant au milieu des mots sont ajoutés avec le préfixe "##".
3. Calculer la fréquence de chaque symbole (caractère ou sous-mot) du corpus.
4. Calculer la fréquence de chaque paire de symboles consécutifs.
5. Calculer le score de chaque paire.

6. Sélectionner la paire ayant le score maximal.
7. Fusionner cette paire : si le second élément de la paire commence par "##", ce préfixe est supprimé avant la fusion. Le résultat de la fusion devient un nouveau symbole.
8. Ajouter le nouveau symbole au vocabulaire.
9. Mettre à jour le corpus en remplaçant chaque occurrence de la paire par sa fusion.
10. Répéter les étapes précédentes jusqu'à atteindre la taille de vocabulaire souhaitée ou jusqu'à ce qu'il n'existe plus de paires à fusionner.
11. Le vocabulaire appris est utilisé pour encoder du texte. Pour chaque mot, WordPiece recherche le plus long préfixe présent dans le vocabulaire. Le reste du mot est ensuite traité de la même manière jusqu'à ce que le mot soit entièrement tokenisé.

3.1.3 Illustration à travers un exemple.

```
|w| w := WordPiece new.
w fromText: 'FRED FED TED BREAD, AND TED FED FRED BREAD'.
```

Vérification du corpus.

Après avoir tokenisé notre texte, nous obtenons ce changement :

```
corpus
#('T' '##E' '##D') 2
#('B' '##R' '##E' '##A' '##D') 2 #('F' '##E' '##D') 2
#('F' '##R' '##E' '##D') 2
#('A' '##N' '##D') 1
```

Lorsque nous examinons le système, nous obtenons les éléments suivants :

- Un vocabulaire : '##D' 'F' '##A' '##R' '##,' 'T' 'B' '##N' '##E' 'A'
- Un corpus : comme présenté ci-dessus
- Une liste de paires consécutives avec leurs
- Une liste de paires consecutives avec leurs scores

```
voc
'##D' 'F' '##A' '##R' 'T' 'B' '##N' '##E' 'A'
corpus
#('T' '##E' '##D') 2
#('B' '##R' '##E' '##A' '##D') 2
#('F' '##E' '##D') 2
#('F' '##R' '##E' '##D') 2
#('A' '##N' '##D') 1
```

pairs

```
#('##E' '##A') 2
#('F' '##E') 2
#('A' '##N') 1
#('F' '##R') 2
#('T' '##E') 2
#('##E' '##D') 6
#('##N' '##D') 1
#('B' '##R') 2
#('##A' '##D') 2
#('##R' '##E') 4
```

letter frequency

```
'##D' 9
'##E' 8
'##R' 4
'##A' 2
'F' 4
'B' 2
'A' 1
'##N' 1
'T' 2
```

scores

```
#('##N' '##D') (1/9)
#('F' '##R') (1/8)
#('##R' '##E') (1/8)
#('##E' '##D') (1/12)
#('B' '##R') (1/4)
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)
#('A' '##N') 1
```

Nous observons que la paire ayant le score maximale est : $\#('A', '##N')$. Cette paire sera la seule qui sera fusionnée. Après sa fusion cette paire donne le symbole 'AN'.

Itération 1.

Examinons la situation après la première fusion.

w mergeOneStep

- Nous voyons que la paire la plus fréquente $\#('A' \ '##N')$ a été fusionnée. Cela signifie que :
- Sa fusion a été ajoutée au vocabulaire : nous pouvons maintenant voir 'AN' comme un nouvel élément.
 - Le corpus a été mis à jour afin de refléter la présence de nouvel élément du vocabulaire : $\#('A' \ '##N' \ '##D')$ est désormais représenté par $\#('AN' \ '##D')$;
 - toutes les paires ont été recalculées à partir de la version mise à jour du corpus. Ainsi, la paire la plus fréquente devient maintenant $\#('B' \ '##R')$.

voc

'##D' 'F' '##A' '##R' 'T' 'B' '##N' 'AN' '##E' 'A'

corpus

$\#('T' \ '##E' \ '##D')$ 2

$\#('B' \ '##R' \ '##E' \ '##A' \ '##D')$ 2

$\#('F' \ '##E' \ '##D')$ 2

$\#('F' \ '##R' \ '##E' \ '##D')$ 2

$\#('AN' \ '##D')$ 1

pairs

$\#('##E' \ '##A')$ 2

$\#('F' \ '##E')$ 2

$\#('F' \ '##R')$ 2

$\#('T' \ '##E')$ 2

$\#('##E' \ '##D')$ 6

$\#('AN' \ '##D')$ 1

$\#('B' \ '##R')$ 2

$\#('##A' \ '##D')$ 2

$\#('##R' \ '##E')$ 4

letter frequency

'##D' 9

'AN' 1

'##E' 8

'##R' 4

'##A' 2

'F' 4

'B' 2

'T' 2

scores

$\#('F' \ '##R')$ (1/8)

$\#('##R' \ '##E')$ (1/8)

$\#('##E' \ '##D')$ (1/12)

$\#('B' \ '##R')$ (1/4)

$\#('AN' \ '##D')$ (1/9)

```
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)
```

Itération 2.

L'algorithme répète le même processus. Ainsi, la fusion 'BR' est ajoutée au vocabulaire. Le corpus et les paires sont ensuite mis à jour.

voc

```
'BR' '##D' 'F' '##A' '##R' 'T' 'B' '##N' 'AN' '##E' 'A'
```

Corpus

```
#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('F' '##R' '##E' '##D') 2
#('AN' '##D') 1
```

pairs

```
#('##E' '##A') 2
#('F' '##E') 2
#('F' '##R') 2
#('T' '##E') 2
#('##E' '##D') 6
#('AN' '##D') 1
#('##D' '##,') 1
#('BR' '##E') 2
#('##A' '##D') 2
#('##R' '##E') 2
```

letter frequency

```
'##D' 9
'AN' 1
'##E' 8
'##A' 2
'BR' 2
'F' 4
'##R' 2
'T' 2
```

scores

```
#('F' '##R') (1/4)
#('##R' '##E') (1/8)
#('BR' '##E') (1/8)
```

```
#('##E' '##D') (1/12)
#('AN' '##D') (1/9)
#('F' '##E') (1/16)
#('##A' '##D') (1/9)
#('##E' '##A') (1/8)
#('T' '##E') (1/8)
```

Itération 3.

Voici l'itération suivante. Le resultat de la fusion 'FR' de la paire #('F' '##R') a été ajoutée à la vocabulaire.

voc

```
'BR' '##D' 'F' '##A' '##R' 'FR' 'T' 'B' '##N' 'AN' '##E' 'A'
```

corpus

```
#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('AN' '##D') 1
#('FR' '##E' '##D') 2
```

Pairs

```
#('##E' '##A') 2
#('F' '##E') 2
#('FR' '##E') 2
#('T' '##E') 2
#('##E' '##D') 6
#('AN' '##D') 1
#('BR' '##E') 2
#('##A' '##D') 2
```

letter frequency

```
'##D' 9
'AN' 1
'##E' 8
'##A' 2
'BR' 2
'F' 2
'FR' 2
'T' 2
```

scores

```
#('FR' '##E') (1/8)
#('AN' '##D') (1/9)
#('##A' '##D') (1/9)
```

```

#('##E' '##A') (1/8)
#('T' '##E') (1/8)
#('F' '##E') (1/8)
#('##E' '##D') (1/12)
#('BR' '##E') (1/8)

```

Itération 4.

Voici l'itération suivante. La fusion 'FRE' de #('FR' '##E') est ajoutée à la liste des fusions. Elle apparaît en première position uniquement à cause du tri utilisé pour l'affichage, qui est le même que celui utilisé pour sélectionner la paire la plus fréquente. Cela n'a pas d'impact sur l'algorithme : d'autres paires de même fréquence auraient pu être choisies.

voc

```
'BR' '##D' 'F' '##A' '##R' 'FR' 'T' 'B' 'FRE' '##N' 'AN' '##E' 'A'
```

corpus

```

#('T' '##E' '##D') 2
#('F' '##E' '##D') 2
#('BR' '##E' '##A' '##D') 2
#('AN' '##D') 1
#('FRE' '##D') 2

```

pairs

```

#('##E' '##A') 2
#('F' '##E') 2
#('T' '##E') 2
#('##E' '##D') 4
#('FRE' '##D') 2
#('AN' '##D') 1
#('BR' '##E') 2
#('##A' '##D') 2

```

letter frequency

```

'##D' 9
'AN' 1
'##E' 6
'##A' 2
'BR' 2
'F' 2
'FRE' 2
'T' 2

```

scores

```

#('BR' '##E') (1/6)
#('AN' '##D') (1/9)

```

```
#('##A' '##D') (1/9)
#('##E' '##A') (1/6)
#('T' '##E') (1/6)
#('F' '##E') (1/6)
#('##E' '##D') (2/27)
#('FRE' '##D') (1/9)
```

Jusqu'à ce qu'a un certain seuil.

L'algorithme se déroule comme décrit précédemment jusqu'à ce que le score de chaque paire devienne inférieur à un seuil défini. On obtient alors la situation suivante.

voc

```
'BR' 'BREA' 'TE' 'FRED' '##N' '##E' 'TED' 'FR' 'A' 'AN' 'BRE' 'BREAD' 'FED' 'F' '##A'
```

corpus

```
#('AN' '##D') 1
#('FED') 2
#('FRED') 2
#('BREAD') 2
#('TED') 2
```

pairs

```
#('AN' '##D') 1
```

letter frequency

```
'FRED' 2
'BREAD' 2
'FED' 2
'TED' 2
'AN' 1
'##D' 1
```

scores

```
#('AN' '##D') 1
```

Utilisation du vocabulaire pour encoder du texte.

La logique de cette étape est la suivante :

Le texte est d'abord découpé en une collection ordonnée de mots M . Soit T la liste des tokens du texte et t la liste des tokens associés à un mot. Au début, T est vide. Pour chaque mot du texte, t est initialisée à vide.

Supposons que le mot courant à tokeniser soit m .

1. On recherche m dans le vocabulaire.

— Si m appartient au vocabulaire, il est ajouté à t .

- Tous les éléments de t sont ensuite ajoutés à T .
 - Le mot suivant dans M devient alors le nouveau mot courant à tokeniser, et le processus recommence à l'étape 1.
2. Si m n'appartient pas au vocabulaire, on recherche le plus long préfixe de m présent dans le vocabulaire.
 - (a) Si un préfixe valide est trouvé :
 - ce préfixe est ajouté à t ;
 - le reste du mot est préfixé par le symbole spécial **##** ;
 - ce reste devient alors le nouveau mot courant m à tokeniser.
 - (b) Si aucun préfixe valide n'est trouvé :
 - le symbole spécial **<UNK>** est ajouté à T ;
 - la tokenisation du mot courant est arrêtée ;
 - le processus passe au mot suivant dans M .
 3. Ce processus est répété jusqu'à ce que le mot soit entièrement tokenisé ou qu'aucun préfixe du mot courant ne soit trouvé dans le vocabulaire.

Exemple : imaginons que nous voulons encoder le texte 'FRED EATS BREAD'. Lors de la phase de découpage du texte en séquences de mots, nous obtenons :

#('FRED' 'EATS' 'BREAD')

Ensuite, nous demandons à l'algorithme d'encoder le texte et nous obtenons la liste de tokens suivante :

```
w encode: 'FRED EATS BREAD'
> an OrderedCollection('FRED' '<UNK>' 'BREAD')
```

Les mots FRED et BREAD sont directement retournés par WordPiece, car ils appartiennent au vocabulaire appris durant l'entraînement.

En revanche, le mot EATS est remplacé par le symbole spécial **<UNK>**, car aucun préfixe valide de ce mot (EATS, EAT, EA ou E) n'existe dans le vocabulaire.

Observation de l'algorithme.

WordPiece et BPE réalisent des opérations similaires durant l'entraînement. Cependant, contrairement à BPE qui sélectionne la paire la plus fréquente, WordPiece calcule un score pour chaque paire de symboles avant de choisir celle à fusionner. Cela nécessite également le calcul de la fréquence de chaque symbole du corpus.

Comme BPE, WordPiece ne fusionne qu'une seule paire à chaque itération. Après chaque fusion, le corpus est mis à jour, puis les fréquences des symboles, les fréquences des paires et les scores des paires sont recalculés.

Le prétraitement du texte et la phase d'encodage diffèrent également entre BPE et WordPiece. WordPiece utilise notamment le symbole spécial `##` pour représenter les symboles (caractères ou sous-mots) situés au milieu d'un mot.

Lors de l'encodage d'un mot, WordPiece n'utilise pas une liste de règles de fusion pour encoder un texte. Il utilise principalement le vocabulaire appris durant l'entraînement pour tokeniser les mots. Il recherche itérativement le plus long préfixe du mot présent dans le vocabulaire. Si aucun préfixe valide n'est trouvé pour le mot courant ou pour le reste du mot à tokeniser, le mot entier est remplacé par le symbole spécial `<UNK>`.

3.2 Implémentation.

L'objectif de ce chapitre est d'implémenter WordPiece en Pharo. Le détail sur le fonctionnement de WordPiece est décrit dans la section **3.1.2**.

3.3 Présentation des algorithmes

3.4 Comparaison théorique

Chapitre 4

Implémentation

4.1 Implémentation de BPE

4.2 Implémentation de WordPiece

4.3 Implémentation de Unigram

Chapitre 5

Résultats et Analyse

5.1 Expérimentations

5.2 Comparaison des performances

Chapitre 6

Conclusion et Perspectives

Annexe A

Annexes

Bibliographie

- [1] A. Kathuria, J. Chaudhary *Emerging Paradigms and Challenges in Multidisciplinary Education and Research* , Iterative International Publisher, 2024, pp. 100-101.
- [2] Choudhary A, Pamidimokkala S, R. K and R. M. B *Impact of Natural Language Processing models on diagnosis and decision-making in healthcare, business, education, and sports : a review*, Front. Artif. Intell, 8 :1706369, 5 february 2026, doi : 10.3389/frai.2025.1706369.
- [3] A. Kathuria, J. Chaudhary *Emerging Paradigms and Challenges in Multidisciplinary Education and Research* , Iterative International Publisher, 2024, pp. 103.
- [4] Daniel Jurafsky, James H. Martin *Speech and Language Processing*, 3rd edition, draft version, 2025, pp. 158
- [5] Ian Goodfellow, Yoshua Bengio, Aaron Courville *Deep Learning*, MIT Press, 2016, pp. 373